

Assignment Report

Name: Asser Mohammed Khairallah

Group Code: GIZ5_SWD5_S2

Assignment: Assignment 02 - LINQ (Language-Integrated Query)

Part 01: Theoretical Questions

1. What is the fundamental difference between Deferred Execution and Immediate (Eager) Execution in LINQ?

- **Deferred Execution:** Operators like `Where`, `Select`, and `OrderBy` do not run when the query is declared. They only build an iterator object (a state machine) that stores the source reference and the delegate logic. The query executes only when the result is enumerated (e.g., via `foreach`, `GetEnumerator().MoveNext()`, or a conversion operator). Each enumeration re-evaluates the query against the current state of the source, so changes to the source between declaration and enumeration are reflected in the result.
- **Immediate (Eager) Execution:** Operators like `ToList()`, `ToArray()`, `ToDictionary()`, `Count()`, `Sum()`, `First()`, and `Max()` force the query to run right away and materialize the result into memory (or return a single scalar value). The result is a snapshot, so later changes to the source do not affect it.

2. How does LINQ utilize Extension Methods under the hood to operate on collections?

- LINQ operators are static extension methods defined in the static class `System.Linq.Enumerable` (for `IEnumerable<T>`) and `System.Linq.Queryable` (for `IQueryable<T>`). An extension method is compiled as a normal static method whose first parameter is marked with `this`, so `source.Where(x => ...)` is rewritten by the compiler into `Enumerable.Where(source, x => ...)`.
- Because the first parameter is `IEnumerable<T>`, every collection that implements this interface (arrays, `List<T>`, `Dictionary<K,V>`, etc.) gets the LINQ operators without modifying the original type. Each operator returns a new `IEnumerable<T>`, which allows method chaining. The lambda arguments are passed as delegates (`Func<T, bool>`, `Func<T, TResult>`), or as expression trees in the case of `IQueryable<T>`.

3. Why are Anonymous Types heavily used in conjunction with LINQ Projection (Select operator)?

- Projection often needs only a subset of properties from the source type (e.g., `Id` and `Name` out of 20 properties), or a shape that does not match any existing class. Anonymous types allow the compiler to generate a temporary, immutable, read-only class on the fly (with `Equals`, `GetHashCode`, and `ToString` implemented), so the developer does not have to declare a dedicated DTO class for every query.
 - Since the type name is generated by the compiler and is not accessible in code, anonymous types are scoped to the method where they are created and are consumed using `var`. This keeps the query result strongly typed at compile-time while reducing boilerplate and unnecessary memory usage from loading unneeded columns or properties.
-

Part 02: Additional Concepts

1. Briefly explain the difference between Fluent Syntax and Query Syntax in LINQ.

- **Fluent Syntax (Method Syntax):** Uses chained extension method calls with lambda expressions, e.g., `list.Where(x => x > 5).Select(x => x * 2)`. It supports all LINQ operators, including those with no query keyword equivalent (such as `Take`, `Skip`, `Count`, and `Distinct`).
- **Query Syntax:** Uses SQL-like keywords (`from`, `where`, `select`, `orderby`, `group`, `join`), e.g., `from x in list where x > 5 select x * 2`. It is more readable for complex operations like joins and grouping. The compiler translates query syntax into fluent syntax at compile-time, so both produce identical IL and have the same performance.

2. Summarize the role of the `var` keyword when evaluating complex LINQ queries and why it is necessary.

- `var` tells the compiler to infer the variable's static type at compile-time from the right-hand side expression. It is necessary for LINQ because query results often have complex generic types (e.g., `IEnumerable<IGrouping<string, Employee>>`) or are collections of anonymous types, whose names are generated by the compiler and cannot be written explicitly in code. The variable remains strongly typed (not dynamic), so IntelliSense and compile-time type checking still work.